



BMG5109 Data Science for Engineers

Ali Abbas PhD. Peng.

aabbas@uottawa.ca



School of Information Technology and Engineering
University of Ottawa



Large Language Model (LLM)

- **LLM is an artificial intelligence model designed to:**
 - Understand human language
 - Generate text
 - Perform reasoning-like tasks
 - ... etc.
- **LLMs are trained on massive text datasets.**





Large Language Model (LLM)

- **Enable natural language interaction**
- **Power chatbots and AI assistants**
- **Support reasoning and planning**
- **Serve as the cognitive and reasoning engine of AI agents.**





What LLMs Can Do

- **Answer questions**
- **Summarize documents**
- **Translate languages**
- **Write code**
- **Reasoning**
- **Assist decision making**
- **Generate Contents**
- **..... Etc.**





What LLMs Cannot Do

- It understand, BUT, do not understand like humans
- Can hallucinate
- Do not verify facts automatically
- Require external grounding





'Large' Model

- **High representational capacity**
- **Larger models capture more complex patterns.**



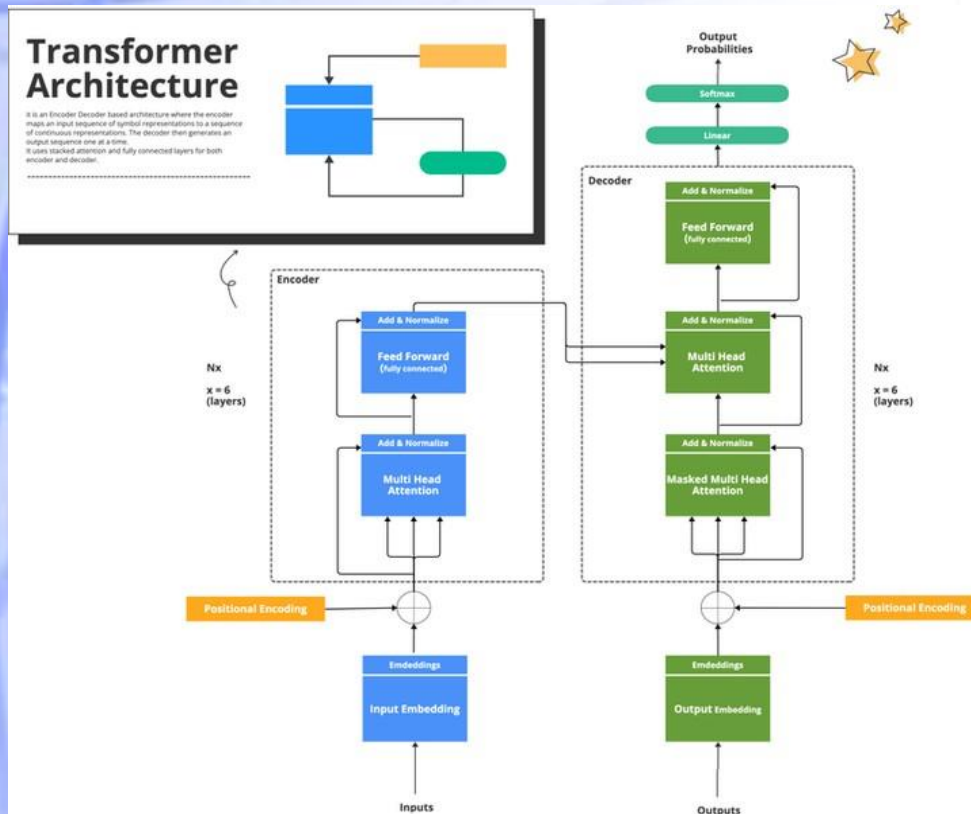


'Large' Model

- Trained on huge datasets
- Millions, billions then trillions of parameters



➤ Transformer is the backbone for these models; : [1] Attention is all you Need.....





Architecture of Transformer Models

The Backbone of new LLMs: **Transformer Architecture.**

At the heart of modern LLMs lies the transformer architecture, a breakthrough in AI that has revolutionized how machines understand and generate human language

Key Mechanism - Self-Attention: [1] Attention is all you Need.....

Transformers utilize a mechanism known as 'self-attention' to process words about all other words in a sentence, enabling the model to understand context (semantic and syntactic in language.)

Benefits of Transformer Architecture:

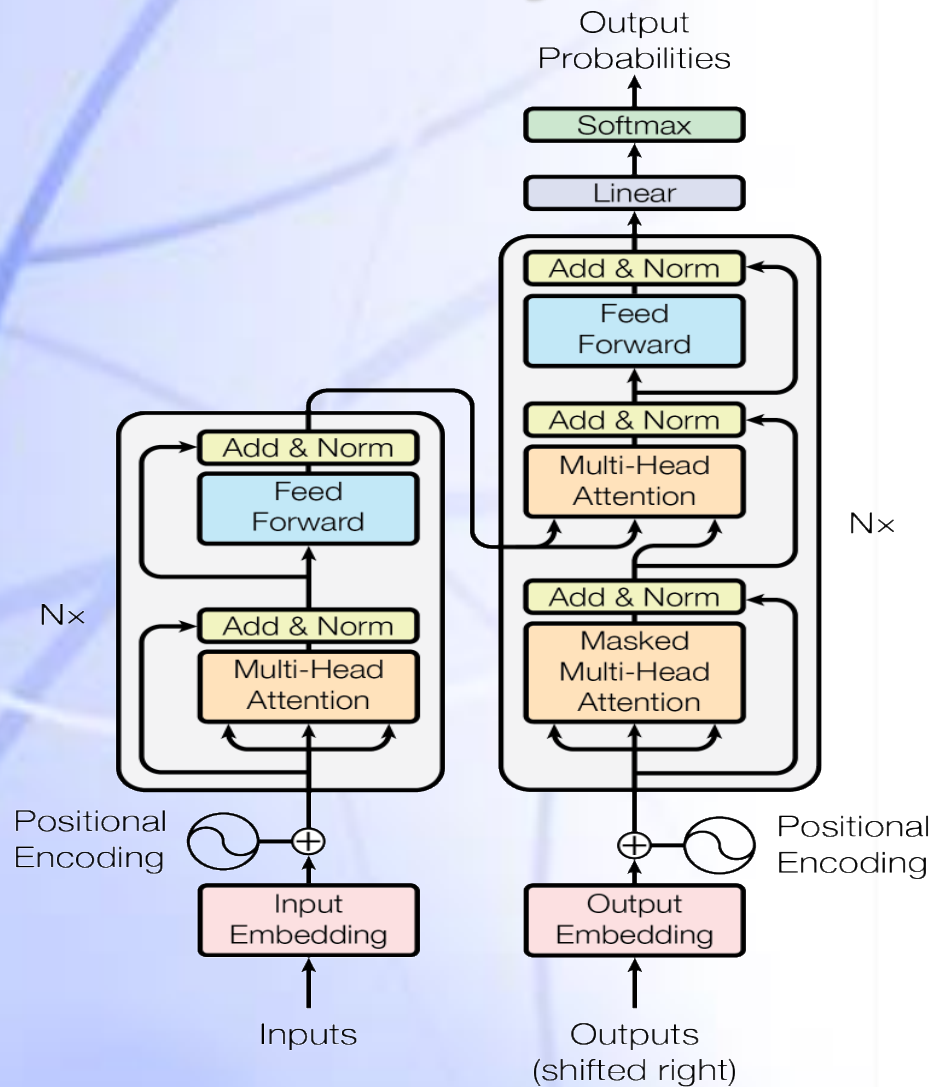
Scalability: Designed to handle vast amounts of data, making them ideal for training on large datasets.

Parallel Processing: Unlike previous sequential models RNN and LSTM, transformers process all words in a sentence simultaneously, significantly speeding up the learning process, hence, its efficient modeling of long-text





Attention is all you Need [1]





Architecture of Transformer Model

- The RNN models with attention mechanisms saw significant improvement in their training and prediction.
- The introduction of transformers by the Google Brain team in 2017 is perhaps one of the most important inflection points in the history of LLMs. A transformer is a deep learning model that adopts the self-attention mechanism and processes the entire input all at once:
- As a significant change to the earlier RNN-based models, transformers do not have a recurrent structure.
 - With sufficient training data, the attention mechanism in the transformer architecture alone can match the performance of an RNN model with attention.
 - Another significant advantage of using the transformer model is that they are more parallelizable and require significantly less training time, this is exactly the sweet spot we require to build LLMs on a large mass of text-based data with available resources.





Architecture of Transformer Models

- Many ANN-based models for natural language processing are built using encoder-decoder architecture. For instance, seq2seq is a family of algorithms originally developed by Google. It turns one sequence into another sequence by using RNN with LSTM or GRU.
- The original transformer model also used the encoder-decoder architecture. The encoder consists of encoding layers that process the input iteratively, one layer after another. The decoder consists of decoding layers that do the same thing to the encoder's output:
- The function of each encoder layer is to generate encodings that contain information about which parts of the input are relevant to each other. The output encodings are then passed to the next encoder as its input. Each encoder consists of a self-attention mechanism and a feed-forward neural network.
- Further, each decoder layer takes all the encodings and uses their incorporated contextual information to generate an output sequence. Like encoders, each decoder consists of a self-attention mechanism, an attention mechanism over the encodings, and a feed-forward neural network.





Transformer Architecture

- **Most LLMs use the Transformer architecture as the base but they elaborated beyond that.**
- **Key components:**
 - Self-attention
 - Feedforward layers
 - Layer stacking





Tokenization

- **Text is split into tokens (word or sub-word).**
 - Example:
 - 'Artificial Intelligence' → ['Artificial', 'Intelli', 'gence']





Transformers – Tokenization

Token ID

original
text

"hello world!"



tokens

['hello', 'world', '!']



token
IDs

[7592, 2088, 999]





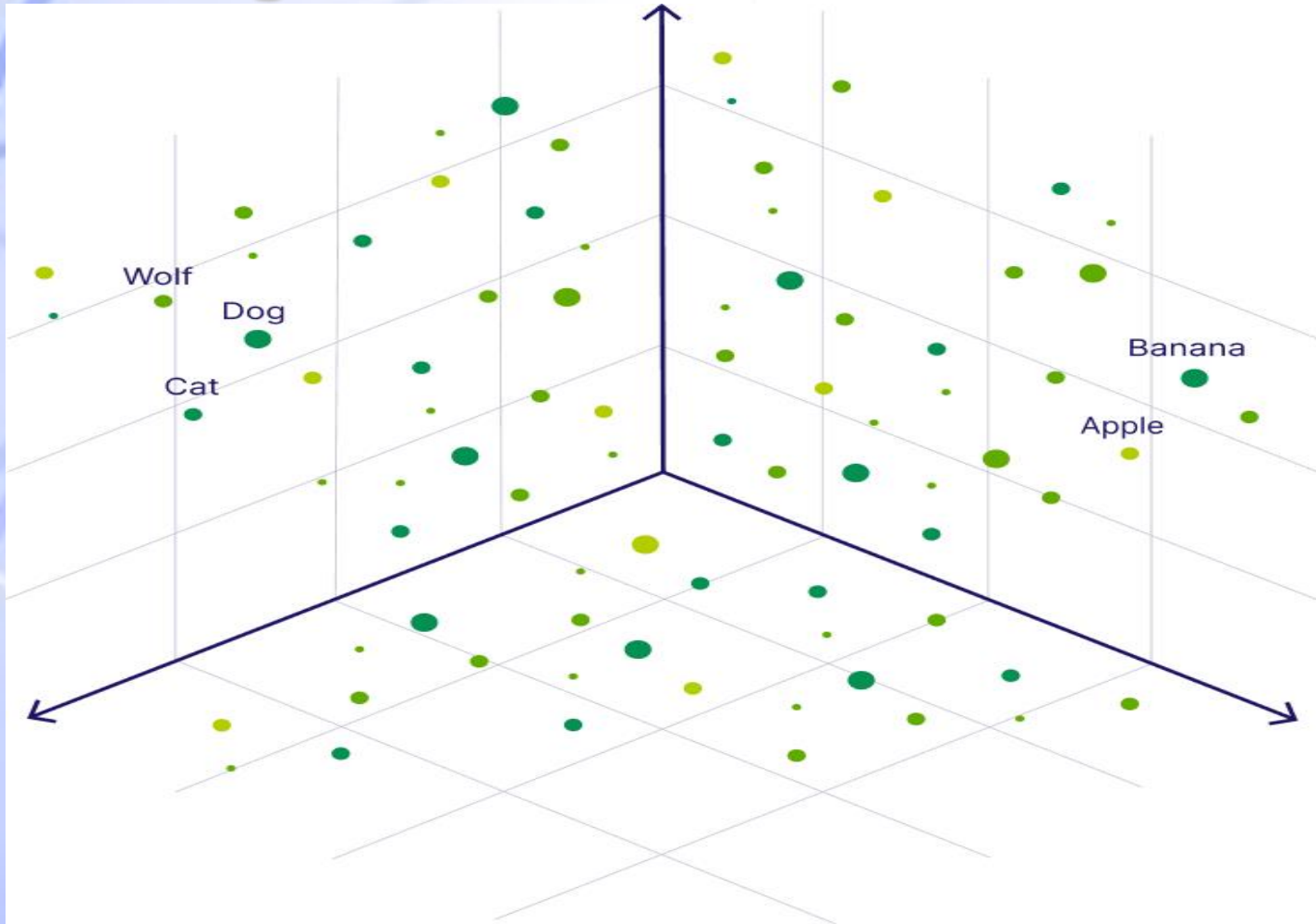
Embeddings

- **Tokens are converted into numerical vectors.**
- **Capture semantic meaning**
- **Similar words have similar vectors**
- **Embeddings are the model's language representation.**





Transformers – High dimensional space Embedding





Transformers – High dimensional space

Positional Embedding

Sequence	Index of token, k	Positional Encoding Matrix with $d=4$, $n=100$			
		$i=0$	$i=0$	$i=1$	$i=1$
I	0	$P_{00}=\sin(0)$ = 0	$P_{01}=\cos(0)$ = 1	$P_{02}=\sin(0)$ = 0	$P_{03}=\cos(0)$ = 1
am	1	$P_{10}=\sin(1/1)$ = 0.84	$P_{11}=\cos(1/1)$ = 0.54	$P_{12}=\sin(1/10)$ = 0.10	$P_{13}=\cos(1/10)$ = 1.0
a	2	$P_{20}=\sin(2/1)$ = 0.91	$P_{21}=\cos(2/1)$ = -0.42	$P_{22}=\sin(2/10)$ = 0.20	$P_{23}=\cos(2/10)$ = 0.98
Robot	3	$P_{30}=\sin(3/1)$ = 0.14	$P_{31}=\cos(3/1)$ = -0.99	$P_{32}=\sin(3/10)$ = 0.30	$P_{33}=\cos(3/10)$ = 0.96

Positional Encoding Matrix for the sequence 'I am a robot'

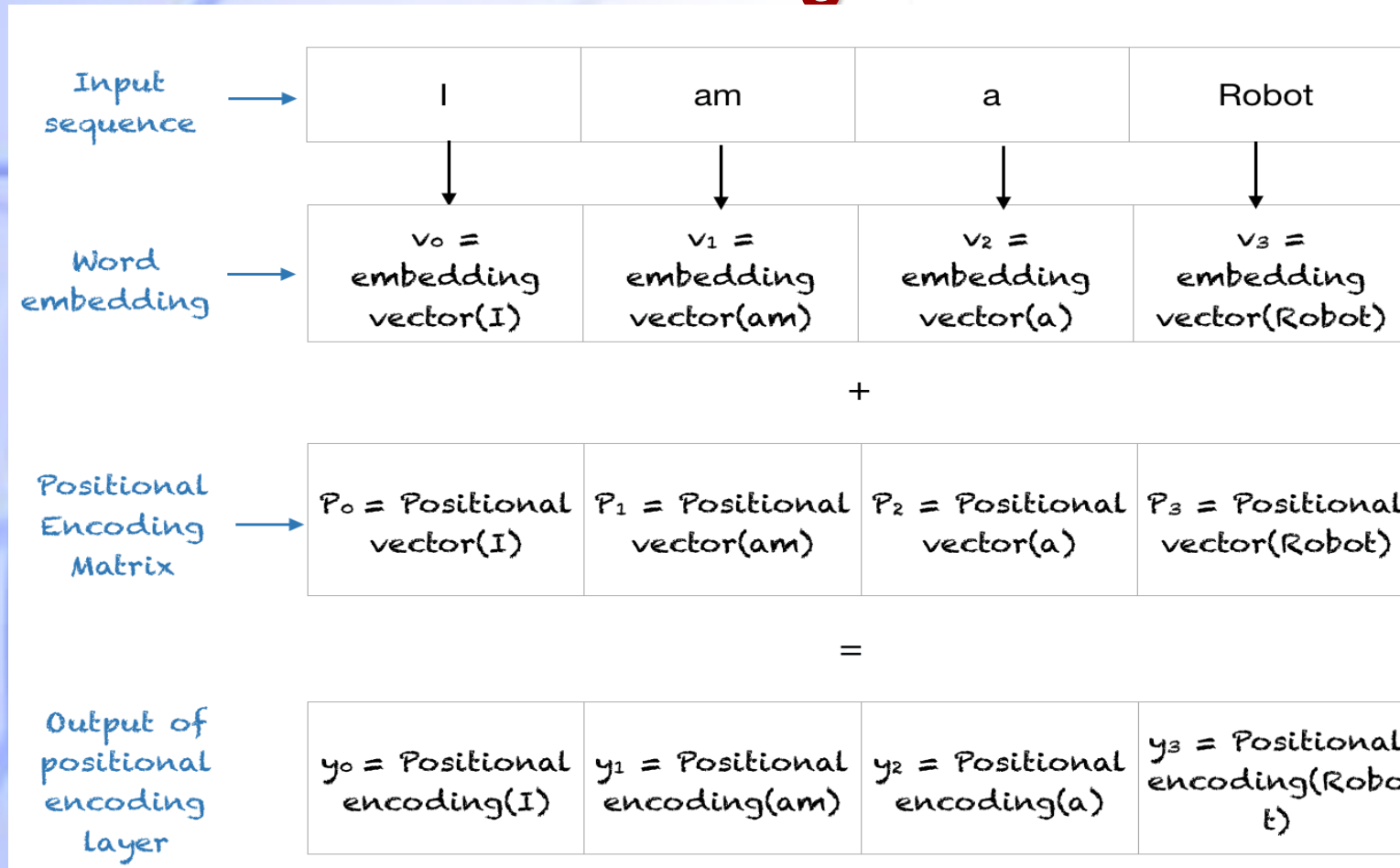
<https://machinelearningmastery.com/a-gentle-introduction-to-positional-encoding-in-transformer-models-part-1/>





Transformers – High dimensional space

Positional Embedding



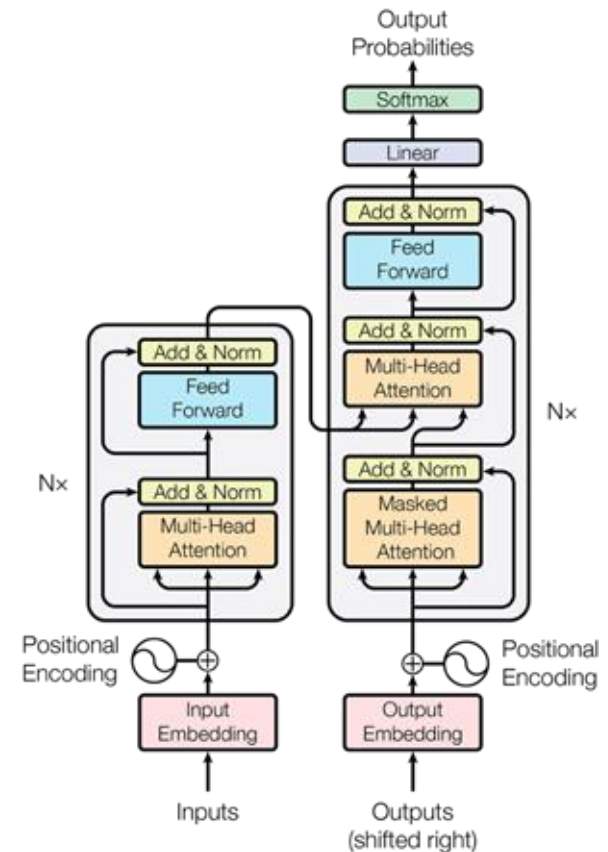
<https://machinelearningmastery.com/a-gentle-introduction-to-positional-encoding-in-transformer-models-part-1/>





Transformer Model Architecture [1]

- Encoder/decoder blocks similar in sizes
- Each block comprises self-attention, optional encoder-attention, and a feed-forward network





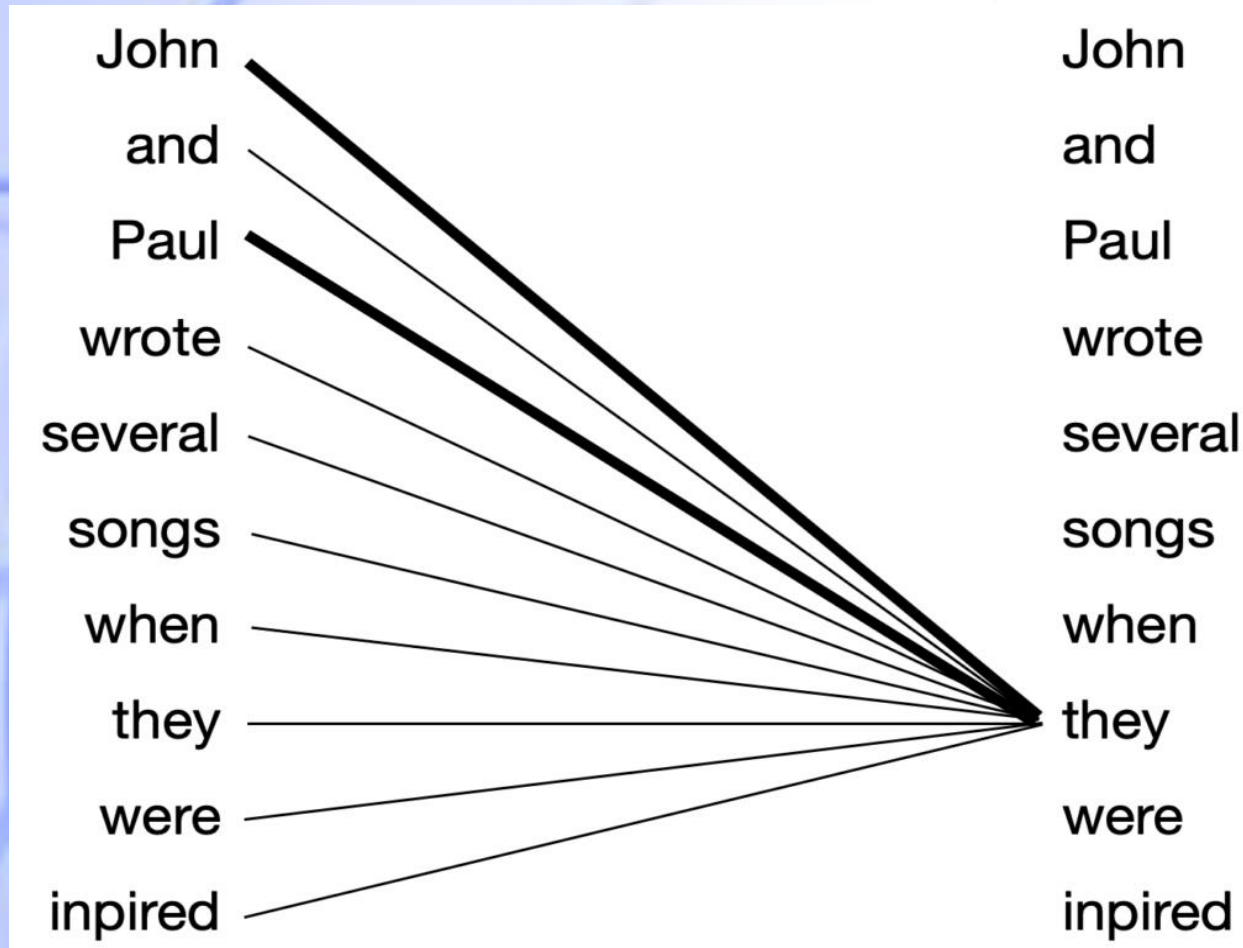
Self-Attention Mechanism

- **Self-attention allows the model to:**
- **Focus on relevant words**
- **Understand long-range dependencies**
- **It answers: 'What matters right now?'**





Self-Attention Mechanism

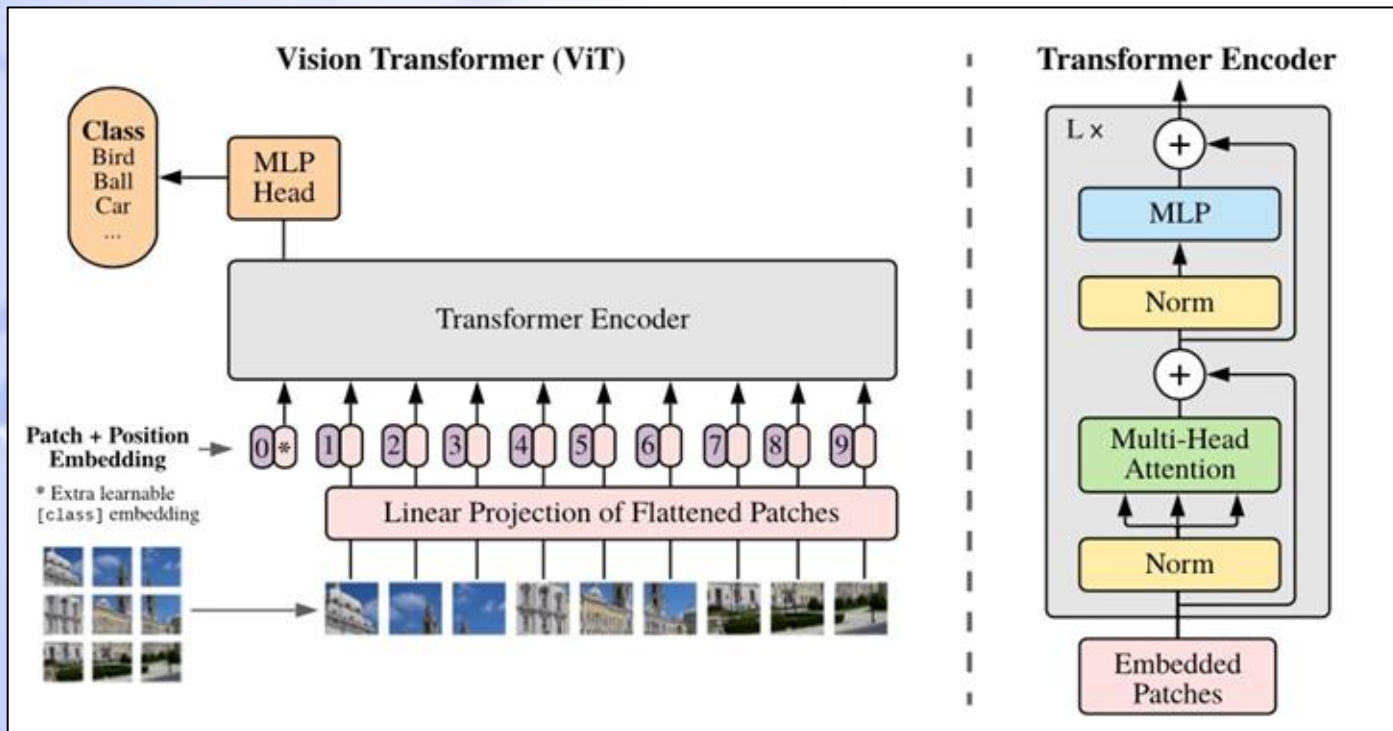


<https://www.alexisalulema.com/2022/08/23/transformers-how-do-they-work-internally/>

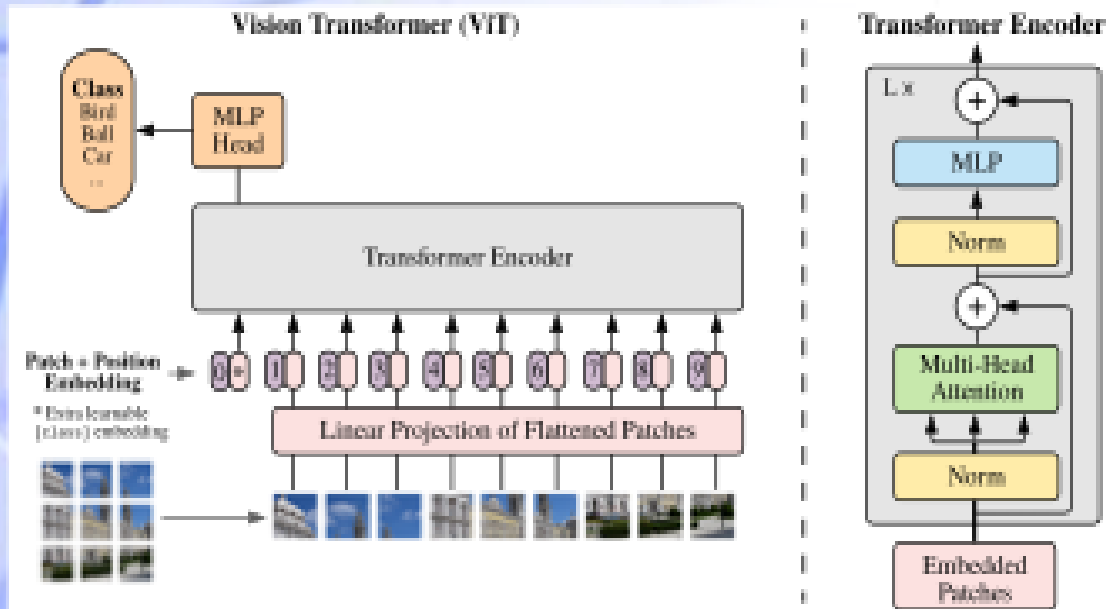




ViT Transformer: Model Architecture



Vision Transformer:



AN IMAGE IS WORTH 16X16 WORDS:TRANSFORMERS FOR IMAGE RECOGNITION AT SCALE [6]



LLMs vs Traditional AI Models

➤ Traditional AI:

- Task-specific

➤ LLMs:

- General-purpose
- Language-driven
- Flexible reasoning
- They predict tokens, not truth
- Best used inside governed systems
- Foundation of modern AI agents





LLMs for AI Agent Systems

- **LLMs act as:**
 - Reasoning engine
 - Planner
 - Language interface
- They require high degree of **governance**.
- AI agents are more “task oriented” that utilize the LLM.





Why Governance Is Needed

- Prevent hallucinations
- Control access to data
- Ensure compliance
- Provide auditability
- Enhance Capacity
- ... etc.





What Is an AI Agent?

An AI agent is an entity that:

- **Perceives its environment**
- **Makes decisions**
- **Acts to achieve goals**
- **Agent = Perception + Decision + Action + Goal**





Why AI Agent Types Matter

- Different problems need different agent designs
- Not all intelligence requires autonomy
- Enterprise systems require governance
- Modern AI combines multiple agent types





Simple Reflex Agents

- Act on current input only
- Use condition–action rules
- No memory or learning
- Examples:
 - Thermostat
 - Rule-based alerts





Model-Based Agents

- **Maintain internal state**
- **Use past information**
- **Handle partial observability**
- **Examples:**
 - Clinical Workflow tracking
 - Patient Monitoring systems





Goal-Based Agents

- **Act to achieve defined goals**
- **Use planning and search**
- **Flexible behavior**
- **Examples:**
 - Task planners
 - Navigation systems





Utility-Based Agents

- **Optimize a utility function**
- **Handle trade-offs**
- **Risk-aware decision making**
- **Examples:**
 - Resource optimization
 - Decision routing





Learning Agents

- **Improve performance over time**
- **Learn from feedback or rewards**
- **Techniques:**
 - Supervised learning
 - Reinforcement learning





LLM-Based Agents

- **Powered by large language models**
- **Capable of reasoning and tool use**
- **Key shift: Language → Action**





Tool-Using Agents

- Can call external tools
- Databases, APIs, calculators
- Loop:
- Reason → Act → Observe → Repeat





Autonomous Agents

- **Minimal human supervision**
- **Self-directed goals**
- **High capability, high risk**
- **Requires strong governance**





Multi-Agent Systems (MAS)

- **Multiple agents interacting**
- **Cooperation or competition**
- **Used in complex AI systems**





Embodied Agents

- **Physical or robotic agents**
- **Sense and act in the real world**
- **Examples:**
 - Humanoid robots
 - Drones





Orchestrated Agent Systems

- **Central controller coordinates agents**
- **Strong governance and auditability**
- **Common in enterprises**





Domain-Specific Agents

- **Data agents**
- **Healthcare diagnostic agents**
- **Risk and compliance agents**
- **Decision agents**





Comparison

- **Reflex → Low autonomy**
- **Goal-based → Medium autonomy**
- **Autonomous → High autonomy**
- **Modern systems combine many agent types**





What Is an Orchestrator?

- **An Orchestrator is a central control component that:**
- **Understands the user request**
- **Breaks it into tasks**
- **Assigns tasks to AI agents**
- **Monitors execution and combines results**





Why Do We Need an **Orchestrator**?

The Orchestrator is the brain that turns multiple AI agents into one intelligent system

- **Multiple AI agents work together**
- **Tasks must be coordinated**
- **Conflicts and duplication must be avoided**
- **Governance, security, privacy, compliance, ethics, and control are required, at least the governance can be common for all agents.**





Orchestrator Core Responsibilities

- **Task planning and decomposition**
- **Agent selection and routing**
- **Context and memory management**
- **Policy and permission enforcement, and other governance requirements.**
- **Result sharing and aggregation**





Multi-Agent System Architecture

- **User → Orchestrator → AI Agents → Data/Tools**
- **The orchestrator sits at the center of the system**
- **Different types of Orchestrators:**
 - Centralized Orchestrator
 - Federated Orchestrator
 - Hybrid Orchestrator





Centralized Orchestrator

- **Single decision point**
- **Strong governance and auditability**
- **Easier security and other regulations enforcement**
- **Common in enterprises**





Federated Orchestrator

- **Local orchestrators per domain**
- **Better scalability**
- **Useful for distributed organizations**





Orchestrator Workflow

- 1. Receive request
- 2. Analyze intent
- 3. Create execution plan
- 4. Call agent(s)
- 5. Collect and validate outputs
- 6. Produce final response





Orchestrator Workflow

Examples:

- **Executive decision systems**
- **Healthcare AI platforms**
- **Autonomous business analytics**
- **Robotics and humanoid AI**





Building Agentic AI for Autonomous Tasks

Agentic AI

- Create a library of the AI agents defined for each & every identified tasks as part of the clinical workflow or clinical reasoning (other industry will be similar)
- This by itself would require variety of different methodologies, Design and implementation approaches, technologies, etc.





Building Agentic AI for Autonomous Tasks

Agentic AI

- The rise of agentic AI / multi-agent systems, which we design them to be autonomous agents (derived from LLMs or other trained AI models) that can perceive, process, plan, act, coordinate, and leverage tools/data to complete tasks.
- When AI agents need to interact with external tools, data sources, workflows, APIs, knowledge bases etc.,
 - **typically, we build each connector individually; this may cause some integration problem.**
- Using MCP (an open protocol standard introduced by Anthropic in late 2024) to unify how AI agents access external context, tools, resources.
- MCP standardizes how AI systems connect to additional tools and data sources.
- Why this matters for our domain (e.g., health informatics, robotics, multi-agents orchestration); easier modularity, tools reuse, composability, governance, scaling.....etc.





Building Agentic AI for Autonomous Tasks

Agentic AI

- An MCP (Model Context Protocol) server is a standardized bridge that connects AI models (LLMs) to external data sources, tools, and systems, enabling them to take real-time actions and access information
- MCP would enables seamless integration between LLM applications (AI Models) and external data sources and tools, It supports *bidirectional*, *tool-invocation* style interfaces for agents;
 - **data sources/tools become accessible by agents via standardized interface.**
- MCP Client which is a component embedded in the agent host (or LLM wrapper) that uses MCP to request tools/data.
- MCP Host (or Host Process) is the environment where the AI models/agents run; orchestrates client, context, tool use.
- MCP Server would includes and implements the protocol for exposing tools, resources, prompts, schemas to the client.





Building Agentic AI for Autonomous Tasks

Agentic AI

MCP Server/Client:

- Deploy (or select) an MCP server implementation; define tool catalog.
- Register initial tools: e.g., EHR query tool, knowledge base tool, scheduling API tool, surgical plan tool, surgery tasks library
- Build or integrate MCP client library in your agent runtime.
- Create tool schemas (input / output definitions).
- Setup security & identity integration.
- Setup logging/monitoring pipeline for tool calls.





Building Agentic AI for Autonomous Tasks

Agentic AI

To enhance the efficiently.....more sophisticated systems

- Federated MCP servers, through which we can build domain-specific servers
 - (e.g., surgery specific, internal hospital, external partner) and agent routing logic.
- Dynamic tool addition and discovery, the server add tools on a runtime and the agent learns new tools at runtime via catalogue.
- Streaming tools & long-running workflows; we could implement streaming transport in MCP.
- Multi-agent orchestration, this will take the agents coordinate to be via MCP and higher-level planner/orchestrator.
- Inter-agent protocol / agent-to-agent tool use beyond MCP (emerging standards).
- Research & standards, we may need to measure agents' performance that was implemented with MCP.





Building Agentic AI for Autonomous Tasks

Agentic AI

eHospital uses multi-agent system:

- Intake/Triage Agent: interacts with patient-facing chatbot, collects symptoms and concluding some health assessment.
- Diagnostics Agent: queries EHR, ask for additional medical test, uses lab/KB tools to identify likely conditions.
- Surgery Planning Agent: uses all the available context processed by other agents.
- Surgical Library Agent: uses all the surgical instruments in the library.
- Scheduling/Orchestration Agent: books follow-up, assigns resources, informs staff.
- Data sources: EHR database, lab results database, MRI imaging, external knowledge base (medical literature).
- Tools: EHRQuery, LabSearch, ImagingAnnotationSearch, LiteratureKBSearch, SchedulingAPI, SurgeryPlanningSearch, SurgicalInstrumentsLookup





Building Agentic AI for Autonomous Tasks

Agentic AI: eHospital uses multi-agent system; need for MCP Implementation

- Deploy MCP server within ehospital network (because PHI). Define tool schemas.
 - **The controller for doctor dashboard needs all the e-Hospital agents. One server; many clients**
- Each agent runs as a client of MCP, **shares session metadata**: user/patient ID, agent role, task ID.
- For example; Triage Agent calls EHRQuery (tool name “ehr_query”, op “search”) via MCP. Receives history.
- Diagnostic Agent calls LiteratureKBSearch (tool “kb_search”, op “query”) via MCP. Receives relevant literature.
- Surgery Planning Agent calls SurgeryPlanning (tool “Surgery_Planning_kb”, op “create_plan”) via MCP. Receives surgery plan to be reviewed by surgeon.
- Surgical Instruments Lib Agent calls SurgicalInstruments (tool “Surgical_Instruments_kb”, op “create_task”) via MCP. Receives surgical task to be reviewed by surgeon.
- Orchestration Agent calls SchedulingAPI (tool “schedule_surgery”, op “create”) via MCP. Receives confirmation.





Building Agentic AI for Autonomous Tasks

Agentic AI: eHospital uses multi-agent system; need for MCP Implementation.

- Audit logs capture each invocation; agent ID, tool, op, latency, result status.
- Access control: only Diagnostic Agent and Scheduling Agent may call “schedule” tool; Triage Agent only “ehr_query” and “kb_search”; different authorization to different agent.
- Observability to monitor tool latency, error rates, number of calls per hour.
- Multi-agent context sharing the outputs, such as the Triage Agent stored in shared memory; Diagnostic Agent reads that and makes next call.
-





Case Study #1

Hospital Readmission Risk Prediction (Healthcare Data Science)

Problem Statement

A large hospital network is experiencing high 30-day patient readmission rates, leading to financial penalties and reduced quality-of-care ratings.

Objective

Develop a predictive analytics system that identifies patients at high risk of readmission before discharge.

Data Available

- Electronic Health Records (EHR)
- Patient demographics
- Diagnosis codes
- Lab test results
- Medication history
- Length of stay
- Previous admissions





Case Study

- **Key Tasks**
 - Data cleaning (missing clinical values, inconsistencies)
 - Feature engineering (comorbidity indices, risk scores)
 - Build classification models (e.g., XGBoost, Logistic Regression)
 - Model interpretability (SHAP, feature importance)
 - Deployment-ready scoring pipeline
- **Expected Outcome**
 - Risk score for each patient
 - Dashboard for clinicians
 - Reduction in readmission rates





References

- [1] Kantardzic, Mehmed. Data Mining. Wiley Professional Development (P&T), 2019.
- [2] Delen, Dursun. Analytics, Data Science, & Artificial Intelligence. (11th Edition). Pearson Education (US), 2019.
- [3] White, Susan. A Practical Approach to Analyzing Healthcare Data. (4th Edition). American Health Information Management Association (AHIMA), 2021.





References

- [4] Attention is all you Need, @inproceedings{Vaswani2017AttentionIA, title={Attention is All you Need}, author={Ashish Vaswani and Noam M. Shazeer and Niki Parmar and Jakob Uszkoreit and Llion Jones and Aidan N. Gomez and Lukasz Kaiser and Illia Polosukhin}, booktitle={Neural Information Processing Systems}, year={2017}, url={<https://api.semanticscholar.org/CorpusID:13756489>} }
- [5] J. Qiu et al., “Large AI Models in Health Informatics: Applications, Challenges, and the Future,” *IEEE Journal of Biomedical and Health Informatics*, Vol. 27, No. 12, pp. 6074-6087, Dec. 2023.
- [6] Large AI Models in Health Informatics: Applications, Challenges, and the Future ; Jianing Qiu , Lin Li , Jiankai Sun , *Graduate Student Member, IEEE*, Jiachuan Peng , Peilun Shi, Ruiyang Zhang, Yinzhaodong, Kyle Lam , Frank P.-W. Lo , Bo Xiao , Wu Yuan , *Senior Member, IEEE*, Ningli Wang, Dong Xu , *Member, IEEE*, and Benny Lo , *Senior Member, IEEE*
- [7] *AN IMAGE IS WORTH 16X16 WORDS: TRANSFORMERS FOR IMAGE RECOGNITION AT SCALE .*
- [8] *AN IMAGE IS WORTH 16X16 WORDS:TRANSFORMERS FOR IMAGE RECOGNITION AT SCALE; Alexey Dosovitskiy*,†, Lucas Beyer*, Alexander Kolesnikov*, Dirk Weissenborn, Xiaohua Zhai*, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, Neil Houlsby,, Google Research, Brain Team {adosovitskiy, neilhoulshby@google.com.*
- [9] [<https://www.intuitive.com/en-us/-/media/ISI/Intuitive/Pdf/xi-x-ina-catalog-no-pricing-us-1052082.pdf>].

